



Approfondimenti su Union Find e Heap

Martedì 12 Maggio 2026

Gestione di insiemi dinamici: union find

- Dato un insieme di N oggetti, supportare in modo efficiente le seguenti operazioni:
 - Connetti due oggetti
 - Esiste un cammino che connette due oggetti?

connect 4 and 3

connect 3 and 8

connect 6 and 5

connect 9 and 4

connect 2 and 1

are 0 and 7 connected? ✗

are 8 and 9 connected? ✓

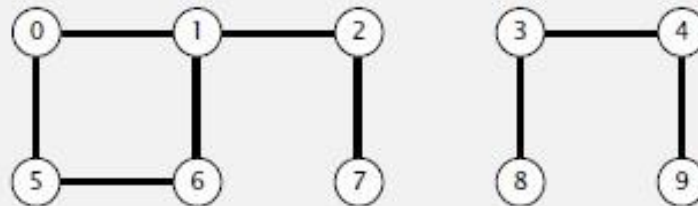
connect 5 and 0

connect 7 and 2

connect 6 and 1

connect 1 and 0

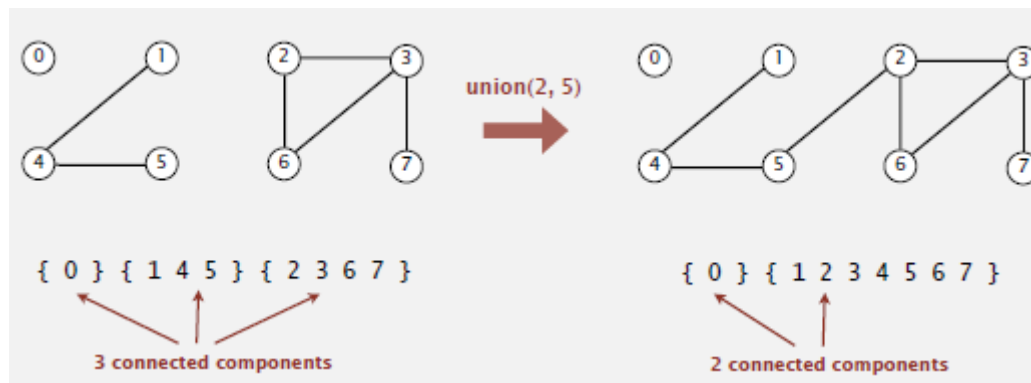
are 0 and 7 connected? ✓



- Rappresento una relazione di equivalenza

Operazioni da implementare

- Find. In quale classe (componente connessa) si trova p?
- Connected. Gli oggetti p e q si trovano nella stessa classe o componente?
- Union. Sostituisco due classi con l'unione delle classi rappresentate da due elementi



Una classe per union-find

```
public class UF
```

UF(int N)

initialize union-find data structure
with N singleton objects (0 to $N-1$)

```
void union(int p, int q)
```

add connection between p and q

```
int find(int p)
```

component identifier for p (0 to $N-1$)

```
boolean connected(int p, int q)
```

are p and q in the same component?

find examines `id[5]` and `id[9]`

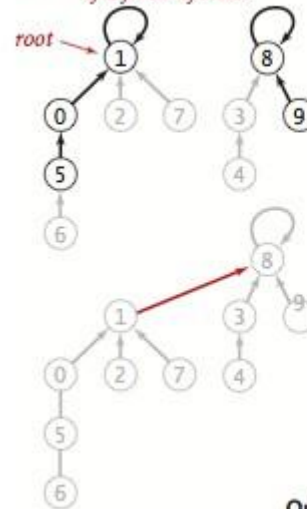
p	q	0	1	2	3	4	5	6	7	8	9
5	9	1	1	1	8	8	1	1	1	8	8

union has to change all 1s to 8s

[illegible]

Quick-find overview

*id[] is parent-link representation
of a forest of trees*



find has to follow links to the root

p	q	0	1	2	3	4	5	6	7	8	9
5	9	1	1	1	8	3	0	5	1	8	8

↑ find(5) is id[id[id[5]]]
 ↑ find(9) is id[id[id[9]]]

union changes just one link

p \ q	0	1	2	3	4	5	6	7	8	9	
5	9	1	1	1	8	3	0	5	1	8	8
1	8	1	8	3	0	5	1	8	8		

Quick-union overview

Quick union

- Struttura dati:

- Integer array `id[]` di lunghezza `N`.

- Interpretazione: `id[i]` è padre di `i`.

	0	1	2	3	4	5	6	7	8	9
<code>id[]</code>	0	1	9	4	9	6	6	7	8	9

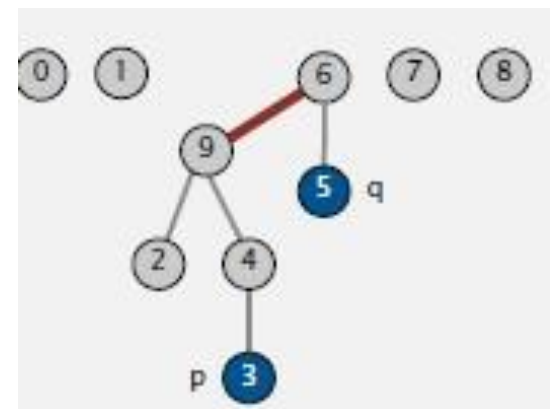
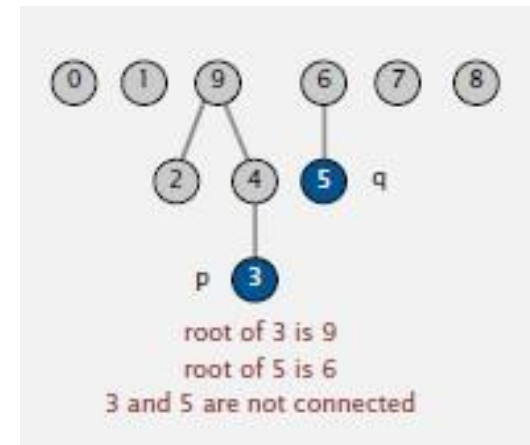
- Find. Chi è la radice di `p`?

- Connected. Due oggetti hanno la stessa radice ?

- Union. Per unire la classe di `p` e quella di `q`, basta far puntare la radice dell'uno alla radice dell'altro

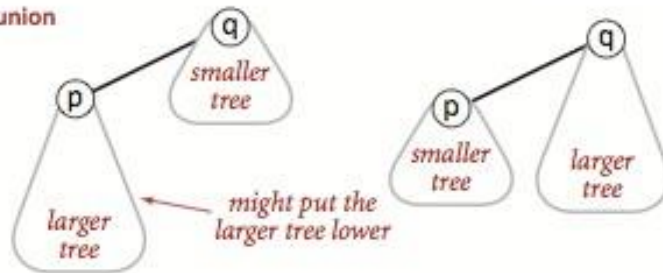
	0	1	2	3	4	5	6	7	8	9
<code>id[]</code>	0	1	9	4	9	6	6	7	8	6

↑
only one value changes

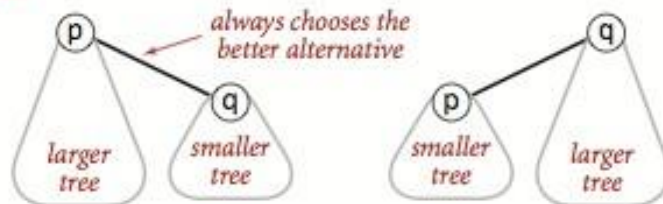


Una classe per union find

quick-union



weighted



Weighted quick-union

algorithm	initialize	union	find	connected
quick-find	N	N	1	1
quick-union	N	N †	N	N
weighted QU	N	$\lg N$ †	$\lg N$	$\lg N$

† includes cost of finding roots

Union Find vs componenti connesse

Caratteristica	Union-Find	Componenti fortemente connesse
Tipo di grafo	Solitamente non orientato	Orientato(diretto)
Scopo	Gestire insieme disgiunti, trovare componenti connesse	Trovare sottografi con mutua raggiungibilità
Approccio	Dinamico (aggiunta archi nel tempo)	Statico (analisi del grafo completo)
Uso tipico	Kruskal, clusterizzazione	Analisi di cicli e raggiungibilità, web graph.

Union Find vs componenti connesse

Caratteristica	Union-Find	Componenti fortemente connesse
Tipo di grafo	Solitamente non orientato	Orientato(diretto)
Scopo	Gestire insieme disgiunti, trovare componenti connesse	Trovare sottografi con mutua raggiungibilità
Approccio	Dinamico (aggiunta archi nel tempo)	Statico (analisi del grafo completo)
Uso tipico	Kruskal, clusterizzazione	Analisi di cicli e raggiungibilità, web graph.

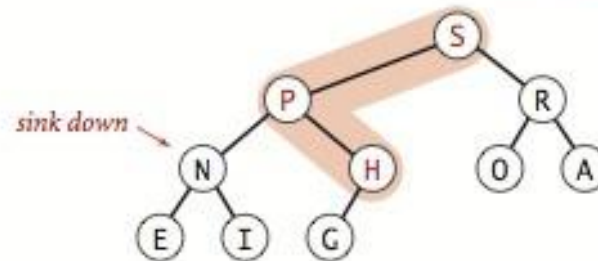
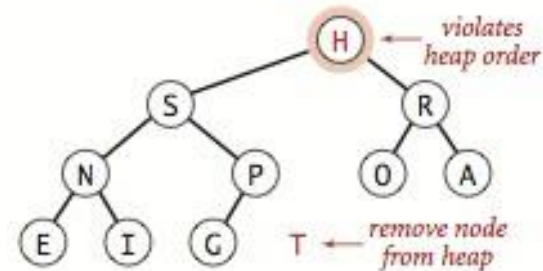
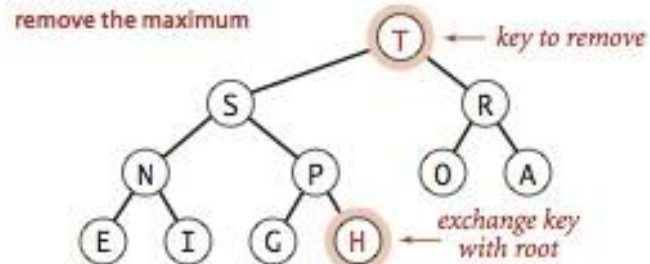
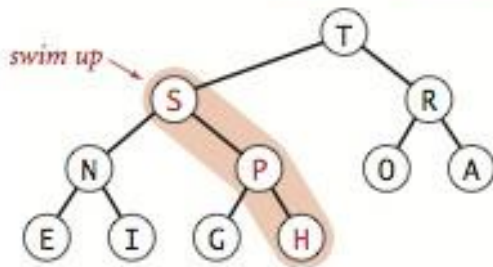
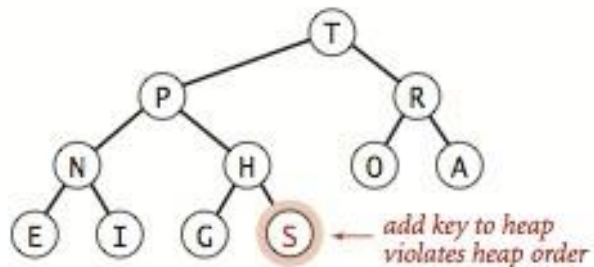
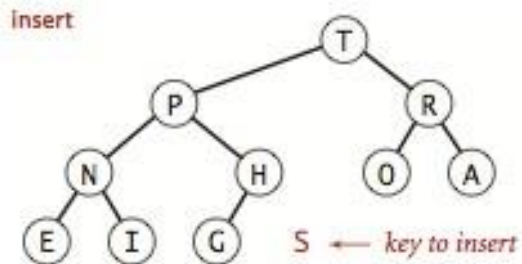
Weighted quick union

- Struttura dati. Uguale alla quick-union, ma usa un extra array $sz[i]$ per contare il numero degli oggetti dell'albero radicato in i .
- Find/connected. Come quick union.
- Union:
 - Collega la radice dell'albero più piccolo alla radice dell'albero più grande
 - Aggiorna $sz[]$ array.
- Esiste una versione ottimizzata che dopo aver esaminato un nodo, pone il suo parent a root (path compression)

Code con priorità (priority queue)

- Le operazioni da rendere efficienti sono estrai il minimo (o il massimo) e inserisci un nuovo elemento.
- Una implementazione efficiente è realizzata tramite heap.
- In un heap binario, gli elementi sono archiviati in un array in modo tale che sia garantito che ciascuna chiave sia più grande (o più piccola) o al più uguale alle chiavi che si trovano in altre due posizioni specifiche. A turno, ciascuna di queste chiavi deve essere più grande di due chiavi e così via. Questo ordine è realizzabile attraverso un albero binario in cui la chiave di ogni nodo è maggiore (o minore) o al più uguale alle chiavi dei due figli (se presenti).
- **Il valore massimo (o il minimo) è posizionato alla radice.**
- Un heap binario può essere rappresentato come un albero binario completo rappresentato attraverso un array
- Il padre di un nodo in posizione k è in posizione $k/2$; e viceversa i due figli di un nodo k si trovano in posizione $2k$ e $2k + 1$.

Operazioni di insert e remove maximum



Heap operations

Coda a priorità indicizzata

- ***Index priority queue.*** In molte applicazioni può essere utile consentire di riferirsi ad item che si trovano già nella coda a priorità. Si associa un unico indice ad ogni item.

- Mantiene un array pq che memorizza l'indice dell'elemento i-esimo nell'array keys.

- Ciò significa che il minimo è `keys[pq[1]]`

```
public class IndexMinPQ<Item extends Comparable<Item>>
```

```
    IndexMinPQ(int maxN)           create a priority queue of capacity maxN  
                                   with possible indices between 0 and maxN-1
```

```
    void insert(int k, Item item)   insert item; associate it with k
```

```
    void change(int k, Item item)   change the item associated with k to item
```

```
    boolean contains(int k)         is k associated with some item?
```

```
    void delete(int k)              remove k and its associated item
```

```
    Item min()                      return a minimal item
```

```
    int minIndex()                  return a minimal item's index
```

```
    int delMin()                    remove a minimal item and return its index
```

```
    boolean isEmpty()               is the priority queue empty?
```

```
    int size()                      number of items in the priority queue
```

Esercizi

1. Implementare il minimum spanning tree, utilizzare i test sul sito <https://algs4.cs.princeton.edu/43mst/> Verificando la complessità confrontando all'aumentare dei nodi.
1. Modificando uno degli algoritmi studiati, aggiungere un metodo che trovi il maximum spanning tree di un grafo pesato non orientato. (usando Union-find